INSPIRE AI

STAND-UP

Standup Meeting



© 2026 Inspire | Confidential

Share one event and choose one next action

Objective · what happened	Describe one event or result from the previous training day.
Reflective · how you felt	Name a feeling. Share only what you choose to share.
Interpretive · what it means	Explain the learning you take from that event.
Decisional · what next	State one action you will take into the rest of the day.

© 2026 Inspire | Confidential

INSPIRE AI

REVIEW

Review the Day 11 Concept Map and Choose a Question



© 2026 Inspire | Confidential

Share a Checked Link and an Open Question

Keep the submission fixed	Each member shares their graded map or their part of the group map. Leave the work and score unchanged.
Read one link aloud	Read both concepts and the linking words. Show the Day 11 slide or file you used to check the link. Peers give one specific comment.
Last 5 min	Choose a question that could change scope or acceptance. Record who can answer it and what the answer would change.

© 2026 Inspire | Confidential

INSPIRE AI

LECTURE

End-to-End AI4SE Development Overview



© 2026 Inspire | Confidential

The whole method on one page before we take it apart

PROCESS	P0 Project baseline establishment Baseline Lead · Spec Architect	P1 Intent Discovery and Proposal Intent Discovery Lead · Intent Owner	P2 G1 Specification definition and alignment Spec Definition Lead · Spec Architect	P3 Design planning and disassembly Design & Breakdown Lead · Spec Architect	P4 G2 Implementation verification and feedback Implementation Lead · Verification Lead	P5 G3 Precipitation archiving and evolution Archive & Evolution Lead · Spec Architect
Human Duties	• Designing Agent-friendly Warehouse Directory Harness • Define constitutional rules (tech stack/architecture/specifications/security/tes • Review and approve directory structure and rules • Formulate quality access control rules and determine "hard constraint" boundaries	• Present business needs and problem background • Determine demand priority and business value • Identify scope boundaries (be clear about what not to do) • Review proposals and decide whether to proceed to the next stage	• Define business rules and domain logic • Confirm the correctness of acceptance scenarios and standards • Supplement edge scenes and abnormal paths • G1 level final approval (human signature)	• Lead technical architecture design and ensure alignment with specifications • Break down Delta Spec into an ordered list of executable tasks • Assess technical risks and decide on solution options • Submit Agentic performance improvement requests to AI Orchestrator	• Critical code review (architecture/security/core logic) • Debugging complex issues and difficult bugs • Check whether the acceptance function meets expectations • G2 level final decision (veto power)	• Confirm the correctness of specification writeback • Submit the repetitive work experience of this CR to AI Orchestrator • Review Warehouse Harness Completeness • G3 level final approval
AI Duties	• Scan the code base to extract architectural patterns and coding habits • Generate the first draft of the Rules rule file • Automatically build the specification library and knowledge base directory structure • Generate workflow templates and checklists	• Analyze requirements to identify ambiguous points and missing information • Proactively ask questions to clarify ambiguities • Assess technical feasibility and scope of impact • Generate a structured first draft of proposal.md	• Generate first draft of Delta Spec • Actively identify ambiguous points and ask questions • Automatically supplement edge scenes and anomalies • Generate Given-When-Then acceptance scenarios and standards	• Generate and compare multiple design options based on specifications • Automatically generate interface definitions and data models • Break down the design into an ordered list of atomic tasks • Generate traceability matrix of tasks and specifications	• Perform coding in task order (TDD mode) • Automatically generate test code • Run validation and generate validation report • Perform Refactor and code optimization	• Automatically merge incremental specs into Living Spec • Detect specification conflicts and propose solutions • Improve knowledge base content and warehouse directory Harness • Generate change summaries and lessons learned

Key Artifacts	`Complete directory structure of the warehouse`
	`Specs/ directory (global specification library)`
	`Knowledge base directory`
	`Rules rule file`
		`proposal.md` `Change directory`
		`Products of the requirements grooming process (user story map, elevator speech, research notes, etc.)`
			`Delta Spec`
			`Acceptance scenarios and standard list`
			`Specification verification report`
				`design.md` `tasks.md`
				`traceability matrix`
					`Implement code` `test code`
					`Verification report`
						`Updated Living Spec`
						`Improved knowledge base`
						`Archive change records`
						`Experience summary`

© 2026 Inspire | Confidential

Six stages, but only one of them runs once per project

Five per change	Stages 1 to 5 run once for every change request, from the proposal to the archive.
One per project	Stage 0 builds the repository, the knowledge base, the specification library and the rules once. Every later change starts from what it left.
Why the plus one	A model that treats the baseline as per-change work rebuilds its own foundations on every request.

© 2026 Inspire | Confidential

Six stages, each one question and one lead role

STAGE	THE QUESTION	LEAD ROLE
P0	What does our project run by?	Spec Architect
P1	What are we doing, and where does the scope end?	Intent Owner
P2 · G1	What counts as right?	Intent Owner + Spec Architect
P3	How is it built?	Spec Architect
P4 · G2	Is it built right?	Verification Lead
P5 · G3	How does the experience stay?	Spec Architect

Read the stages as questions, not as actions. Merge two and the gate between them loses what it blocks.

© 2026 Inspire | Confidential

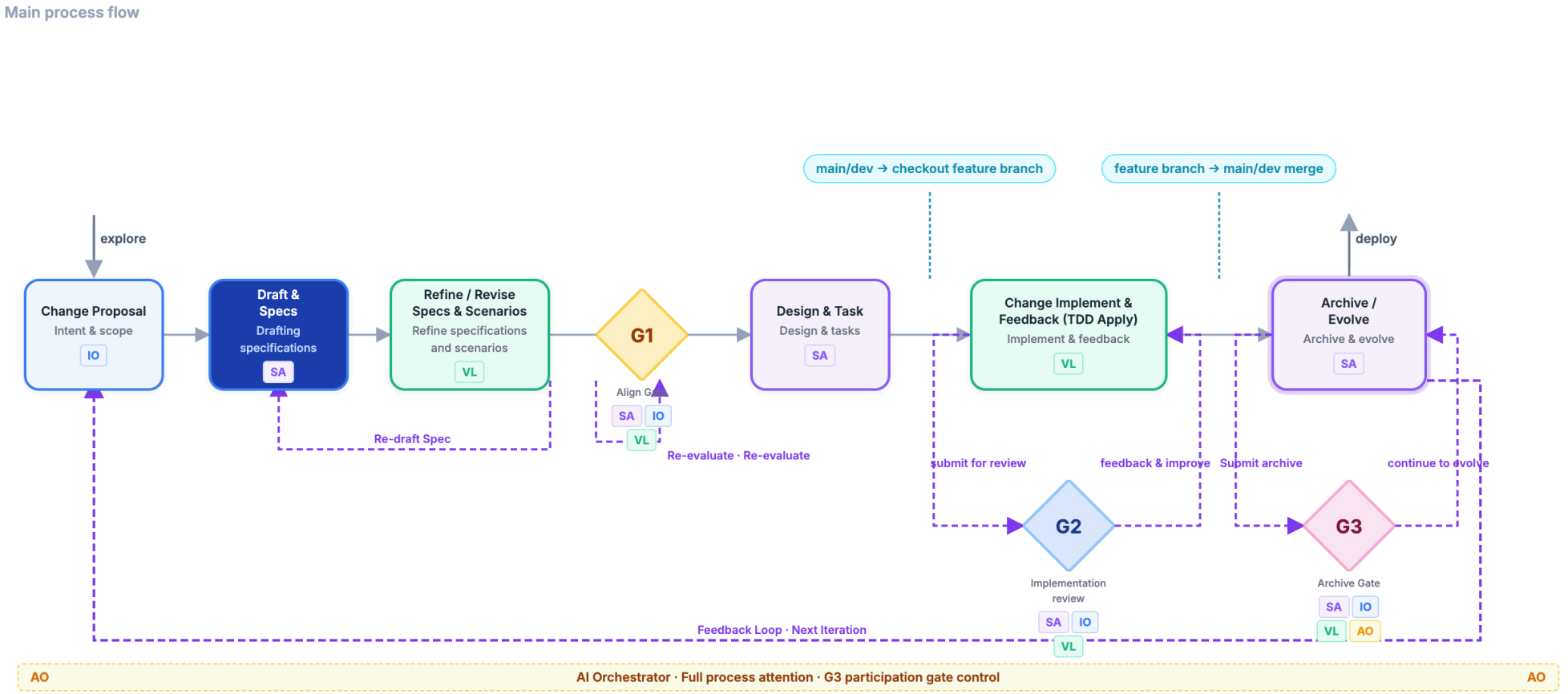
Four roles, each judged by the failure it owns

ROLE	SCOPE OF ACCOUNTABILITY	THE FAILURE IT OWNS
Intent Owner	Business decisions: what to do, what not to do, and priority	We did the wrong thing: the specification was defined wrongly, or we built something we should not have
Spec Architect	Specification quality and system consistency: completeness, architecture decisions, constraint design	The system drifted apart: spec drift, architectural defects, process gaps
AI Orchestrator	How the methodology is implemented in AI: quality and reuse of SKILL, Agent and Harness assets	Our AI assets fell behind: quality, reuse rate or drift rate below the bar
Verification Lead	Completeness of verification: strategy, quality gates, defect interception	We let something through: verification was insufficient and a findable defect shipped

A role that cannot name its failure cannot be held to it in a retrospective.

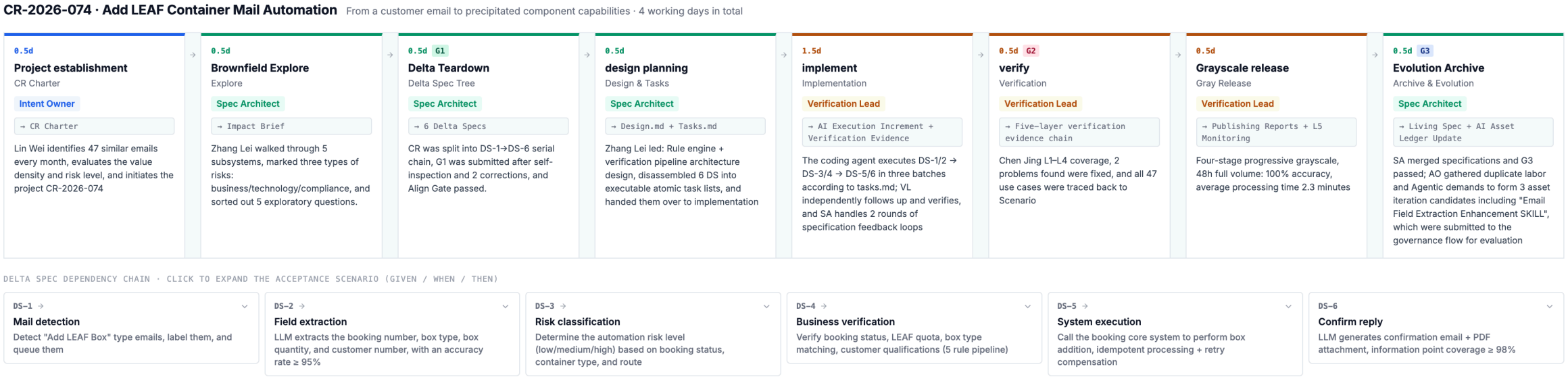
© 2026 Inspire | Confidential

One figure, three ways back



© 2026 Inspire | Confidential

The same figure on one real case: CR-2026-074



© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

LECTURE

Establish the Project Baseline (P0)



© 2026 Inspire | Confidential

P0 builds the repository a coding agent starts from

Before any change starts, the repository holds the directory layout, the knowledge base, the specification library and the rules that govern work, so every later change starts inside the same context instead of from zero.

Harness here means the project runtime: that directory tree, knowledge base, specification library and rule carrier together.

P0 is the only project-level stage. Every other stage runs once per change, on top of what stage 0 left behind.

© 2026 Inspire | Confidential

Two activities decide two different things

ACTIVITY	WHAT IT DECIDES
Design the repository layout	Where knowledge, specifications and code live, and which of them a change reads first
Define the constitution rules	Technology choice, architecture principles, coding standards, security baseline, testing requirements

Both are decisions with an owner. Neither is a document to fill in.

© 2026 Inspire | Confidential

Four artifacts are what remains after those activities

ARTIFACT	WHAT IT HOLDS
Directory structure	The complete layout, including where knowledge and specifications sit
Specification library	The global library of accepted behavior, the basis a change is admitted from
Knowledge base	Verified facts and decisions, kept so the next change stands higher
Rule set	The constitution rules, with one authoritative copy each

The specification library is settled before the code layout: the code tree is where the specification lands, not the other way round.

© 2026 Inspire | Confidential

PRACTICE

Explore the Supplied Baseline Repository



© 2026 Inspire | Confidential

Three boundaries make the observation hold

No installation	The toolchain behind this baseline is not needed to read it today.
No edits	Work in a copy or read-only. Nothing you change today is part of the record.
No gate	Its own quality check is self-imposed. Read what it covered and what it left open; do not grade it.

© 2026 Inspire | Confidential

Read the two entries before you look around

The run table	The steps of the baseline stage and the condition under which each can be skipped.
The recipe	The five steps in order: which one applies first in a brownfield repository, which is optional.
Why it matters	The baseline is a process trimmed to the situation, not a checklist everyone runs to the end.

© 2026 Inspire | Confidential

Column 1: which artifact lives at which path

ARTIFACT	REAL PATH
Specification library	openspec/specs/ and openspec/changes/
Knowledge base	docs/knowledge/
Requirement workspace	requirement-workspace/
Rule carrier	AGENTS.md plus docs/knowledge/rules/
AI tool projection	.qoder/

Write the path, not the artifact name. A cell counts only when the path opens in front of you.

© 2026 Inspire | Confidential

Column 2: three rules, one line of evidence each

Open the index and the schema	The index lists the rules; the schema states what a rule entry must contain.
Classify three	Constitution or lint, using admission against shape.
A third answer is allowed	Cannot tell is acceptable when the reason is written: a rule can describe structure and imply admission at once.

© 2026 Inspire | Confidential

The entry file carries pointers, not rule text

One authoritative copy	The rule body sits in the rule library; the entry file names it by identifier.
A projection is derived	Any tool-specific copy is generated from the body, so a copy is never the authority.
Why it matters	Two copies of a rule disagree the first time one is edited.

© 2026 Inspire | Confidential

Column 3: what our group is missing

Missing cell	Which artifact we do not have at all.
Present but wrong	Which cell exists and we would change, and what it costs us next time.
Rule we never wrote	Which constitution rule our group has never decided, and who could decide it.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Judge a Skill, Then Build a Code-Review Rule Skill



© 2026 Inspire | Confidential

Reusable experience, not a longer Prompt

A Prompt says it this time. A Skill says what to do whenever this kind of task appears.

	PROMPT	SKILL
What it is	A one-time instruction	Settled process knowledge
What it holds	Format, role, limits, this round's context	Trigger, steps, tool preferences, constraints, acceptance
Risk vs value	Grows long, reuses poorly, acceptance scatters	Personal experience becomes a callable agreement

From reminding the model every time to the system following the process on its own.

© 2026 Inspire | Confidential

Why package experience as a Skill

An open format that packages knowledge, process and tools into a capability an Agent can call.

Fills business context	Real work lacks the process knowledge, team conventions and company context the model does not have.
Settles knowledge into a capability	Expert experience, rules and best practices packaged once, reused across many agent products.
Stable and auditable process	Multi-step work fixed into one consistent workflow: less improvisation, more repeatability.

© 2026 Inspire | Confidential

A Skill is a folder with SKILL.md at its core

A light, open packaging format: besides the description and metadata, it can carry scripts, reference material and resources.

```
my-skill/
├── SKILL.md          # required: metadata + instructions
├── scripts/          # optional: executable code
├── references/       # optional: docs / reference knowledge
└── assets/           # optional: templates / images / resources
```

© 2026 Inspire | Confidential

Load on demand: only the complexity the current task needs

Discovery	At startup, only name and description are read.
Activation	The full SKILL.md loads after a task matches.
Execution	Scripts, references and assets are used on demand.

Progressive disclosure: context pays only for the current step.

© 2026 Inspire | Confidential

Six criteria for a good Skill, none of them about length

CRITERION	WHAT A GOOD ONE LOOKS LIKE
Trigger	Says when to call it and when not to
Steps	Someone else can follow the order
Stop	When an input is missing it stops instead of finishing
Artifact	Names where the output goes and what counts as acceptable
Dependencies	Lists the files it reads and what the team still has to supply
Composition	Fits the steps before and after it, and does not claim the whole flow

© 2026 Inspire | Confidential

Today we build one; we do not evaluate, package or align it

No evaluation loop	The full evaluation run and its viewer do not fit in 40 minutes.
No packaging	Nothing is published or bundled today.
No schema alignment	We do not copy the reference repository rule format. Build first, align later.

© 2026 Inspire | Confidential

Write three sentences before you open the tool

What it does	Review one change with our group code review rules.
When to call it, and when not	Which requests belong to it, and which are out of reach.
What it produces, and where	The artifact and its path. If nobody can say, that is a finding.

© 2026 Inspire | Confidential

Draft the contract; keep the rules in a reference file

1 **Feed the group rules in and answer its questions** 10 min

- Boundaries, input and output format, expected artifact.
- An answer nobody in the group can give stays open instead of being invented.
- The rule text never moves into the contract file.

The tool will ask for boundaries, input format and expected artifact. Answer from the group rules, not from memory.

© 2026 Inspire | Confidential

Three resources, then the structure check

RESOURCE	WHAT IT MUST CONTAIN
references/cr-rules.md	Every rule with a number and a source
assets/ review template	The verdict criteria, plus one failing example
The description	When to call the Skill, and when not to

Then run the tool structure check. Structure first, content second: a failing structure means nothing downstream can be trusted.

© 2026 Inspire | Confidential

Three runs that tell you whether it works

Should trigger	Review this change request. Did the description bring the Skill in?
Should not trigger	Write this function for me. Was it kept out?
Missing reference file	Delete the rule file and run again. Did it stop, or did it review anyway?

© 2026 Inspire | Confidential

Record the failures in three categories and mark one to fix

No trigger	The request belonged to it and it was not called.
Unacceptable artifact	It ran, and the output is not something the group would accept.
No stop	An input was missing and it finished anyway.
Mark one	The failure the group most wants to fix, and why that one.

© 2026 Inspire | Confidential

BREAK

Lunch



© 2026 Inspire | Confidential

PRACTICE

Lightning Talk: One Decision and Its Evidence



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Clarify the Intent and Write the Proposal (P1)



© 2026 Inspire | Confidential

Requirement background

A company whose main business is project delivery often has to bring people from inside the company onto new projects. The project lead knows which posts the project needs, which skills, and when the people are needed. The resource coordinator knows the company's people, finds candidates, confirms the arrangement with the person, and then hands the proposed candidates to the project lead for confirmation. Today this work is done mainly through spreadsheets and chat. Post requirements are easily missed, the results of conversations are scattered in different places, and the project lead finds it hard to tell which posts are confirmed and which are still short of people. As projects multiply, people's schedules collide and travel arrangements turn out to be unsuitable. We want to build a small application where the two kinds of user coordinate around the same post requirement and can see the currently confirmed list of people.

© 2026 Inspire | Confidential

Requirement description

A company runs several projects at once and often has to find suitable employees for a new project or for the empty posts of an existing one. It wants a Staffing system for the project lead and the resource coordinator: the project lead raises the post requirement, the resource coordinator selects employees from the company's people pool and recommends them, and after the project lead confirms, both can see the chosen people and the posts that are still short. Finding people for a project has to weigh whether the employee's role, level and skills meet the post requirement, whether the employee's existing and future project arrangements fit this project's dates, and whether the employee accepts the travel the project requires.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

P1 Showcase: Compare Proposals and Name the Gaps



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Generate the P2 Delta Spec and Practice It



© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

P2 Showcase: The Delta and Its Scenarios



© 2026 Inspire | Confidential

PRACTICE

Run the G1 Gate and Sign the Baseline



© 2026 Inspire | Confidential

What G1 confirms

One shared picture of expected behavior for the Intent Owner, the designers and the verifiers, organized by four questions: What does the business do? How does the user work with it? How does it cooperate externally? What level must it run at?

STAGE	MAIN CONFIRMATION	EXAMPLE
G1 · requirement alignment	Business behavior, interaction, external integration and operational quality requirements	Headcount 0 rejects the save; the existing post stays unchanged
P3 · design planning	Internal APIs, data model, module split, technical approach and test design	Fields, error states and data handling of the post update API
G2 · implementation verification	Whether the actual result meets the confirmed requirements	Submit headcount 0: the save is rejected and the data is unchanged

G1 settles the requirements and the acceptance basis; actual results are checked at G2. User-visible results, errors and data changes belong to G1; internal APIs can wait for P3. Even the company's own HR system counts as external.

© 2026 Inspire | Confidential

How to use this reference standard

Select items	Choose the check items that apply to this change and state the check scope explicitly.
Record N/A reasons	For items that do not apply, record the reason, for example: no external system integration this time.
Agree pass conditions	Set the concrete pass conditions from business need and technical feasibility together.
Keep the evidence	Check item by item against the agreed conditions, and keep each result with its basis.

Adjusted scope or pass conditions need a recorded reason and owner confirmation.

© 2026 Inspire | Confidential

Four categories and their artifacts

CATEGORY	WHAT IT COVERS	SUGGESTED CARRIER
Business specification	Goal and scope, roles and permissions, core flows, business rules, data meaning, states and lifecycle	proposal.md for goal and scope; delta spec.md for behavior rules and acceptance scenarios
Interaction experience	Page structure, operation flows, feedback, and the display under each permission and state	A versioned prototype; what it cannot express goes into spec.md
External integration	System boundary, exchanged data, trigger timing, success result, exceptions and the responsible party	integration-contract.md, referenced from spec.md
Operational quality	Performance, availability, compatibility, security, accessibility, recovery, migration and launch	Non-functional requirements and operational constraints in spec.md

Category is not separate files: business specification and quality may share one spec.md; no integration means no contract file. Two review records stay besides: dov-ac-review.md (items, results, evidence, gaps) and g1-decision.md (decisions and versions).

© 2026 Inspire | Confidential

Every category is checked in the same four steps

STEP	HOW
1 · List what must be checked	List the business, interaction, integration and quality requirements item by item so nothing is missed; number the items that need tracking
2 · State which version is checked	Record the document and prototype versions, and keep a link that can retrieve that exact content; after a change, re-check the affected parts
3 · Write the result and its basis	For example: 10 items in total, 8 passed, 2 to be completed. List the gaps and attach the corresponding material
4 · State what is not checked	For what this round does not cover, write the reason and have the owner confirm it; undecided or unchecked items never count as passed

List everything first, then check item by item. What was missed does not count as passed, and every pass states its basis so someone else can re-verify it.

© 2026 Inspire | Confidential

Business specification: B1 to B5

The question: are the rules settled and decidable for any concrete case?

NO.	CHECK	HOW TO CHECK	PASS WHEN
B1	Scope and sources	Map each scope item to a spec; trace each behavior to a source	Every item has a spec; no sourceless behavior; no conflict with an exclusion
B2	Roles and permissions	What each role can do on which data, even logged-out or non-owner	Every case allowed or denied; multi-role merging agreed
B3	Inputs and rules	Meaning, requiredness, range, unit, default, uniqueness	No missing definition; each branch has input and result; no conflicts
B4	States and data	State and data after success, failure, cancel and delete	Actor, condition and result explicit; nothing undecided
B5	Acceptance scenarios	Map rules, permissions and transitions to scenarios	Precondition, input, expected result and data change present

Boundary checks: below, at and above each bound; one representative per legal option.

© 2026 Inspire | Confidential

Interaction experience: U1 to U4

The question: does the user know how to operate, and what happens next?

NO.	CHECK	HOW TO CHECK	PASS WHEN
U1	Flow walkthrough	Walk each flow from entry to goal, recording page, action and result	Every in-scope flow walks through; no step leads nowhere
U2	State display	Normal, empty, loading, success, validation failure, service failure, no permission, read-only	Every applicable state has a screen or note; every inapplicable one a reason
U3	Operation results	Save, cancel, back, delete and retry where applicable	Before and after state, kept input and data change are explicit; nothing undecided
U4	Consistency and confirmation	Compare fields, options, permissions and feedback against the spec	Zero unresolved differences; IO confirmation of a specific version recorded

No real backend is needed: loading and timeout can be simulated or stated.

© 2026 Inspire | Confidential

External integration: I1 to I4

The question: what crosses the boundary, how do the sides cooperate, who owns failures?

NO.	CHECK	HOW TO CHECK	PASS WHEN
I1	Integration scope	External systems, direction, purpose, triggers and responsible party	Every interaction listed; no open boundary or responsibility
I2	Data contract	Field meaning, requiredness, format, units, options and identity mapping	Required fields defined; every interaction has one concrete example
I3	Exceptions and duplicates	Rejection, timeout, duplicate, partial success and out-of-order cases	Every branch has a result and a recovery; retry safety is answered
I4	Version and responsibility	Contract version, the other side's limits, joint confirmations, verification	All confirmations closed; version, owners and method retrievable

The IO confirms semantics and acceptable exceptions; the SA checks the contract.

© 2026 Inspire | Confidential

Operational quality: Q1 to Q4

The question: are the requirements specific enough to measure or check later?

NO.	CHECK	HOW TO CHECK	PASS WHEN
Q1	Applicability	Performance, availability, compatibility, security, accessibility, recovery, migration, launch	Each item applicable or has a written reason
Q2	Measurable definition	Object, condition, metric, pass bar and method per requirement	Pass or fail decidable; fast, stable, secure alone is not enough
Q3	Verification arrangement	Environment, data, tools or manual steps, stage and owner	Every requirement has an executable arrangement
Q4	Exceptions and limits	Unsupported platforms, limits and deferrals, with impact and owner	Every exception confirmed; not yet implemented is not N/A

Targets come from business need and technical feasibility, never from the agent.

© 2026 Inspire | Confidential

Recording results and the G1 decision

RESULT	MEANING
Pass	Evidence shows the project-confirmed pass condition is met
Fail	A rule, input, result or gap contradicts the pass condition
Not verified	Not yet checked, or the evidence is insufficient to judge
N/A	Out of scope this time, with the reason and the confirmation complete
DECISION	**CONDITION**
G1 Pass	All applicable items pass; every N/A is confirmed; nothing undecided, no open artifact difference; the IO and the SA confirmed the same versions
Conditional	Not a pass: record the gaps, their owners, the completion conditions and the scope allowed to continue

A non-author re-checks on the same version; the agent organizes evidence and never approves.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

LECTURE

Explain the Homework and Tomorrow's Entry



© 2026 Inspire | Confidential